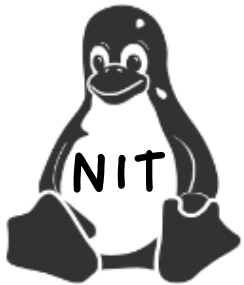


Welcome



Everyone



MASTERING LINUX!

10 - KEYBOARD SHORTCUTS



- 1 Ctrl + A => BOL (beginning of line)
- 2 Ctrl + C => Interrupt a process
- 3 Ctrl + D => exit
- 4 Ctrl + E => EOL (end of line)
- 5 Ctrl + L => This clears the screen
- 6 Ctrl + U => Clears line and takes you to BOL (acts like an eraser)
- 7 Ctrl + Z => STOPS a process (BG %1)

8  -----> scroll history UP

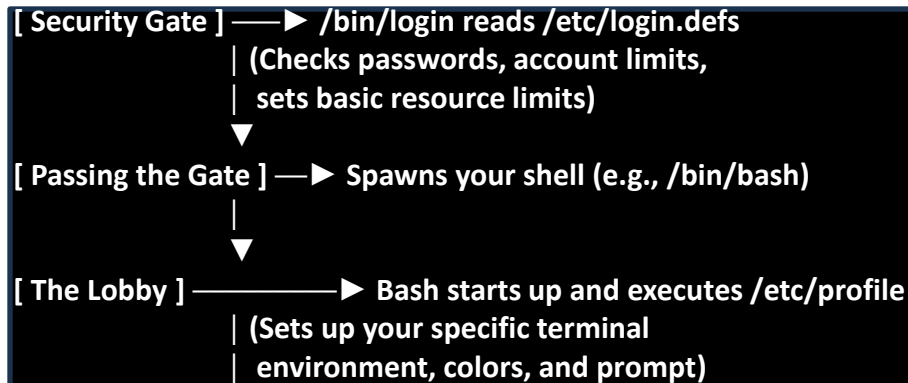
9  -----> scroll history DOWN

10  ----> saves time typing

PRE-INITIALIZATION

The Step-by-Step Handoff

Think of the login process like entering a secure building. The login binary is the security guard at the front gate, and `/etc/profile` is the orientation guide inside the lobby.



- **Authentication:** You provide your credentials. The login binary checks `/etc/passwd`, `/etc/shadow`, and references `/etc/login.defs` to see system-wide parameters (like password expiration rules, or what group IDs are allowed).
- **Dropping Privileges:** The login binary starts out running as root so it can check system files. Once it verifies who you are, it switches its user ID (UID) to *your* user ID for security.
- **Spawning the Shell:** Now that it is safely acting as "you," it looks at your default shell (usually `/bin/bash`) and launches it.
- **Sourcing the Profile:** The newly launched Bash shell looks around, notices it is a brand-new login session, and says, "Okay, time for me to configure the environment." **Only now** does `/etc/profile` get read and executed.

Why doesn't the login binary just set everything?

- Historically, Unix systems were designed with the principle of **separation of concerns**:
- `/etc/login.defs` is for the **operating system's** login utilities to configure account security and initial process boundaries.
- `/etc/profile` is for the **shell** to configure the user's working environment (like command aliases, terminal colors, and custom paths).



USER MANAGEMENT

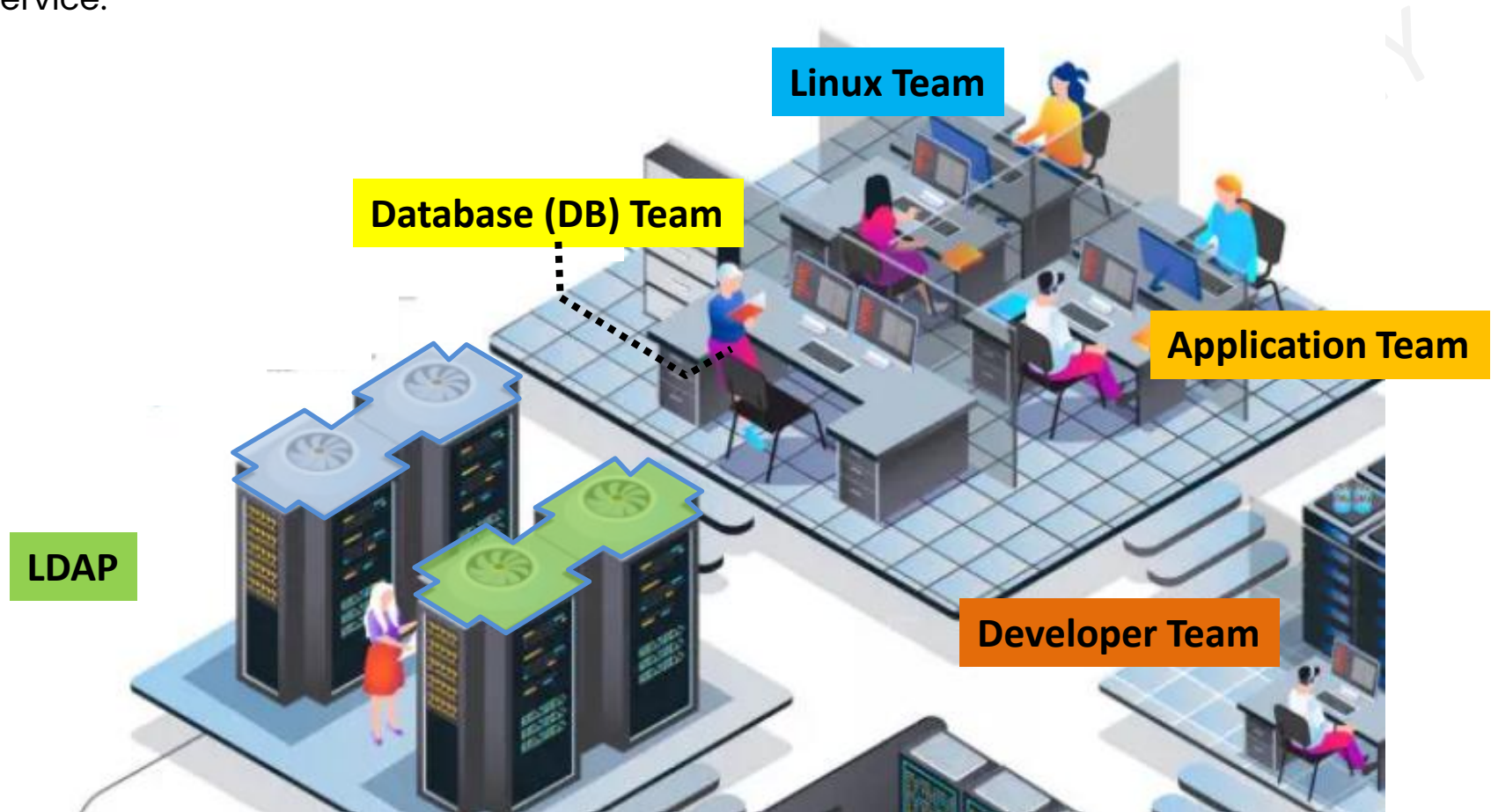
- IAM -

Identity & Access Management

PROPERTY OF V

USER MANAGEMENT OVERVIEW

In companies today **Active Directory (AD)** serves as the backbone for **managing users** and **resources**, while LDAP provides the means for external systems to interact with this directory service.



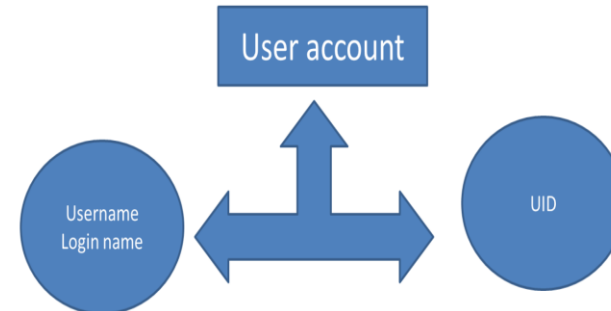
INTRODUCTION

Powered by people, steered by safety™



A user account is a systematic approach to track and monitor the usage of system resources.

1. **Root or Superuser**
2. **Regular or normal user**
3. **Service user account**



Each user account contains two unique identifiers:

Username is used to access the user account. Username is also known as login name.

UID is used to authenticate, track and monitor the activity of user account. Username is used by the user while the UID is used by the system. Typing “**#id john**” will give you UID and GID of User John.

LINUX OS USER

The Standard Linux Operating System User are 5 Type of Users in Linux:

1. System User (This is Root User and the Admin)
2. Normal User (This is a user created by Root User)
3. Network User (this is a remote user that is logs-in Windows Active Directory or LDAP in the case of Linux Users}
4. System User based on an installed application like Oracle etc.
5. Sudo User (normal users who have admin or root privileges.

A	B	C	D	E	F
Type of User	Example	User ID	Group ID	Home Directory	Default Shell
Super User	Root	0	0	/root	/bin/bash
Normal	John, Bob	500-60000	500-60000		
System User	ftp, ssh, apache	1-499	1-499		
Network User	Remote user	same as normal user	same as normal users		

HOW MANY USERS IN THE SYSTEM

#w
#whoami

STEP1: How many users are in the System

\$ uptime

[nitadmin2@VM01 /]\$ w

21:39:49 up 6:30, 3 users, load average: 0.57, 0.32, 0.17

USER	TTY	LOGIN@	IDLE	JCPU	PCPU	WHAT
nitadmin	tty1	15:11	5:20m	0.04s	0.04s	-bash
nitadmin	pts/0	15:13	0.00s	1.35s	0.11s	w
tom	pts/1	21:39	8.00s	0.25s	0.25s	-bash

STEP 2: Who are you?

[nitadmin2@VM01 /]\$ whoami

nitadmin2

Column	Description
USER	Username of the logged-in user.
TTY	Terminal session name (e.g., pts/0 for remote or tty1 for local login). \$tty
FROM	Source IP or hostname of the connection. (- for local logins)
LOGIN@	Login time (HH:MM format) when the user session started.
IDLE	Idle time (how long since the user last interacted with the session).
JCPU	Total CPU time used by all processes attached to this terminal. \$top
PCPU	CPU time used by the current foreground process. \$vmstat
WHAT	Current running command in the user's session.

WHEN DID A USER LAST LOG IN?

#lastlog

```
[nitadmin2@VM01 etc]$ lastlog
```

Username	Port	From	Latest
root	pts/2		Tue Jan 7 02:35:02 -0600 2025
dbus			**Never logged in**
sssd			**Never logged in**
chrony			**Never logged in**
sshd			**Never logged in**
nitadmin2	pts/1	192.168.1.207	Tue Jan 14 02:36:25 -0600 2025
apache			**Never logged in**
bob	pts/0		Wed Jan 1 18:24:11 -0600 2025
alice	pts/1	192.168.1.200	Tue Jan 7 06:26:55 -0600 2025
tom	pts/2	192.168.1.207	Mon Jan 13 22:24:12 -0600 2025

#lastb – bad logins

```
[root@VM01 nitadmin2]# lastb
```

Username	Service	IP Address	Time	Duration
khalid	ssh:notty	192.168.1.207	Tue Jan 14 04:18 - 04:18	(00:00)
vm2	ssh:notty	192.168.1.207	Tue Jan 14 00:52 - 00:52	(00:00)

#last – List of Users Logged-in and Logged out / shutdown

```
[nitadmin2@VM01 log]$ last
```

Username	Port	IP Address	Time	Status
tom	pts/3	192.168.1.207	Tue Jan 14 04:31 - 04:31	(00:00)
alice	pts/2	192.168.1.207	Tue Jan 14 04:14	still logged in
nitadmin	pts/1	192.168.1.207	Tue Jan 14 02:36	still logged in

USER KILL

```
#who -u
#kill -9 PID
```

```
[nitadmin2@VM01 /]$ w
 22:24:21 up  7:15,  4 users,  load average: 0.06, 0.05, 0.06
USER      TTY      LOGIN@  IDLE   JCPU   PCPU WHAT
nitadmin  tty1     15:11   6:05m  0.04s  0.04s -bash
nitadmin  pts/0    15:13   3.00s  1.50s  0.02s w
tom       pts/2    22:24   9.00s  0.07s  0.07s -bash
[nitadmin2@VM01 /]$ who -u
nitadmin2 tty1          2025-01-13 15:11 06:05          708
nitadmin2 pts/0        2025-01-13 15:13 .          1444 (192.168.1.207)
tom       pts/2        2025-01-13 22:24 .          3780 (192.168.1.207)

[nitadmin2@VM01 /]$ sudo kill -9 3780
[sudo] password for nitadmin2:

[nitadmin2@VM01 /]$ w
 22:24:56 up  7:15,  4 users,  load average: 0.04, 0.05, 0.06
USER      TTY      LOGIN@  IDLE   JCPU   PCPU WHAT
nitadmin  tty1     15:11   6:05m  0.04s  0.04s -bash
nitadmin  pts/0    15:13   0.00s  1.50s  0.01s w
```

LINUX OS USER

How do you check details about LINUX OS USER?

```
[root@Practicelab~]# cat /etc/login.defs
```

Min/max values for automatic uid selection in useradd

#

```
UID_MIN          1000
```

```
UID_MAX          60000
```

What if the number of users have to exceed 60,000?

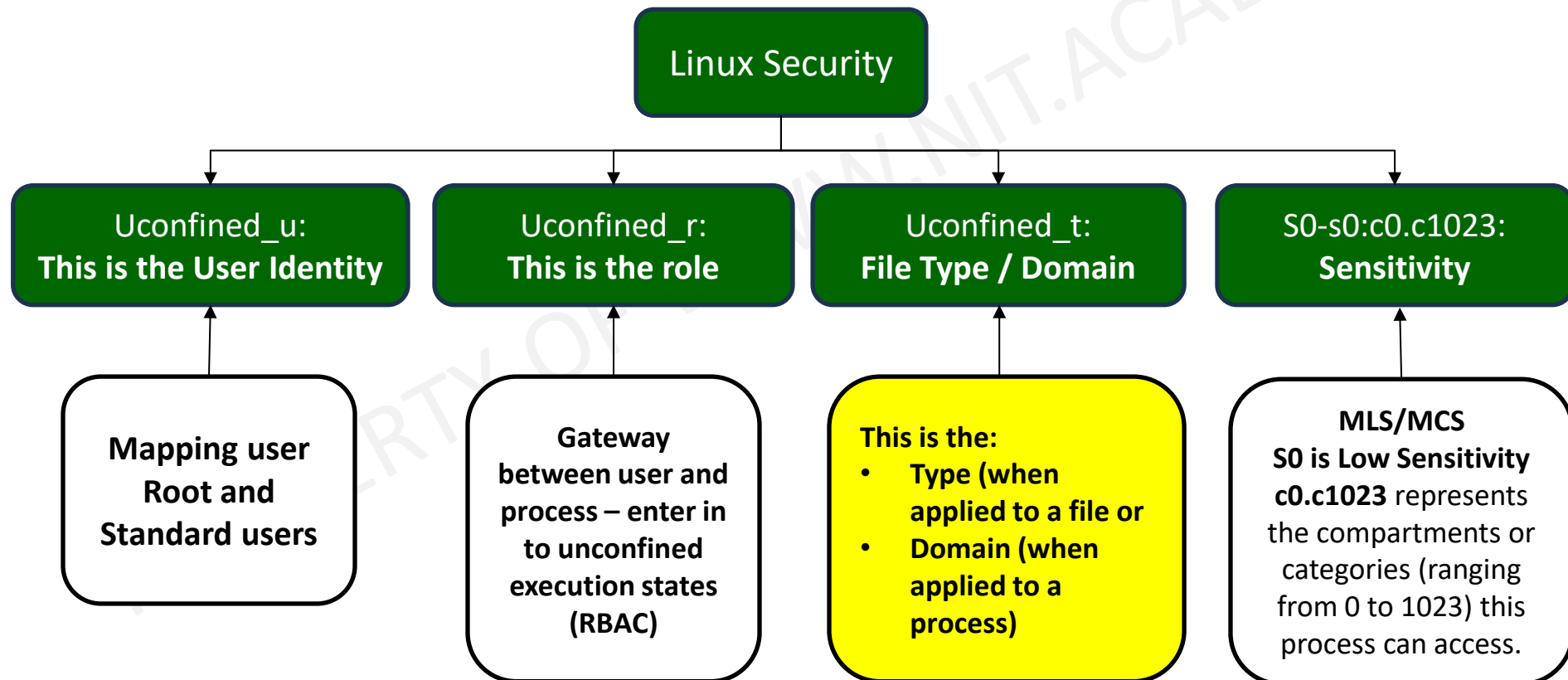
```
[root@Practicelab~]# useradd -u 75000 bill_thrasher
```

SELINUX USER – THE REAL IDENTITY

```
[root@Practicelab ~]# id
uid=0(root) gid=0(root) groups=0(root) context=unconfined_u:unconfined_r:unconfined_t:s0-
s0:c0.c1023
```

Let us understand the four distinct components of “Identity” command

SELinux doesn't care about traditional Linux usernames or UIDs (like **root** or **uid=0**); instead, it reads this specific label to decide what a process is allowed to do.



SELINUX USER

How do you check details about SELinux identity?

```
[root@Practicelab ~]# id -Z
```

```
unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
```

unconfined_u: (User): The SELinux user identity. It has nothing to do with Linux UIDs

unconfined_r: user_r is reserved specifically for restricted, non-administrative SELinux users

unconfined_t: allowing standard Linux DAC (Discretionary Access Control) permissions to take precedence

NOTE: Currently user is mapping to “unconfined_r” meaning they are not restricted to move about the entire Linux File System

SELINUX - PRACTICE LAB

How do you check details about SELinux identity?

[root@Practicelab ~]# **dnf install -y policycoreutils-python-utils**

Installed:

checkpolicy-3.6-1.el9.x86_64	policycoreutils-python-utils-3.6-5.el9.noarch
python3-audit-3.1.5-8.el9.x86_64	python3-distro-1.5.0-7.el9.noarch
python3-libsemanage-3.6-5.el9_6.x86_64	python3-policycoreutils-3.6-5.el9.noarch
python3-setools-4.4.4-1.el9.x86_64	python3-setuptools-53.0.0-15.el9.noarch

Complete!

semanage – SELinux Policy Management Tool

[root@Practicelab ~]# **semanage login -l**

Login Name	SELinux User	MLS/MCS Range	Service
__default__	unconfined_u	s0-s0:c0.c1023	*
root	unconfined_u	s0-s0:c0.c1023	*

Linux Security

Discretionary Access Control
(DAC)

Mandatory Access Control
(MAC)

Role Based Access Control
(RBAC)

DAC vs. MAC in Linux

Discretionary Access Control (DAC)

Owner controls access



In DAC, the **user** holds the keys—Flexible but risky.

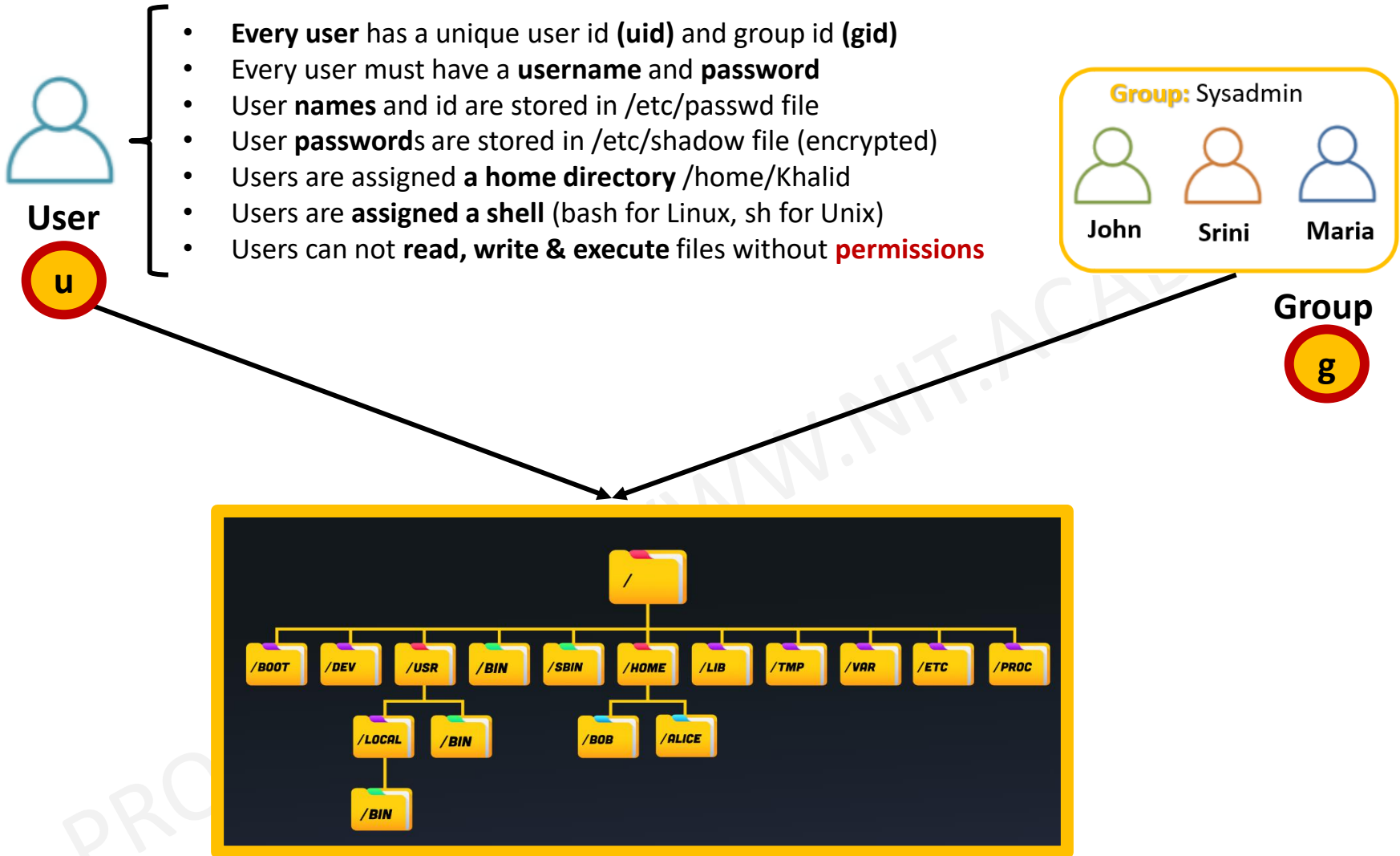
Mandatory Access Control (MAC)

System enforces access



In MAC, the **System** holds the rules ♥ Secure but strict.

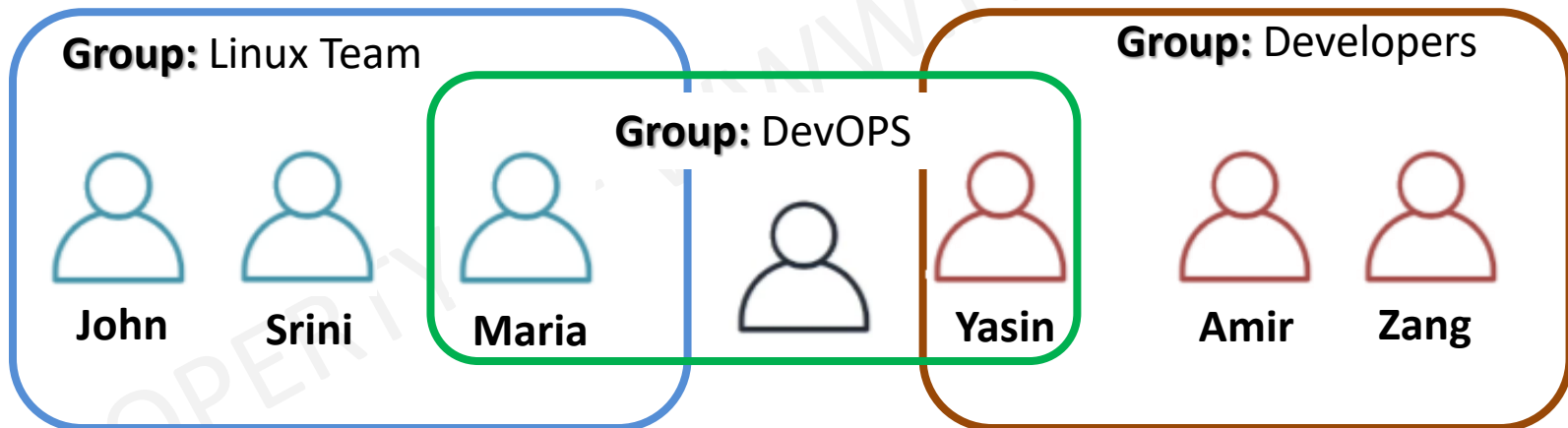
USER MANAGEMENT



NOTE: Every file on the system is owned by a user and associated with a group

LINUX OS USER & GROUPS

1. **Users** are people within an organization and can be grouped
2. **Groups** only contain Users. A user can be added to a maximum of **16 groups**
 - **Primary Group** – all users are part of one **(1) primary group**
 - **Secondary Group** – user(s) can be added to a maximum of **15 secondary groups**



USER ADD



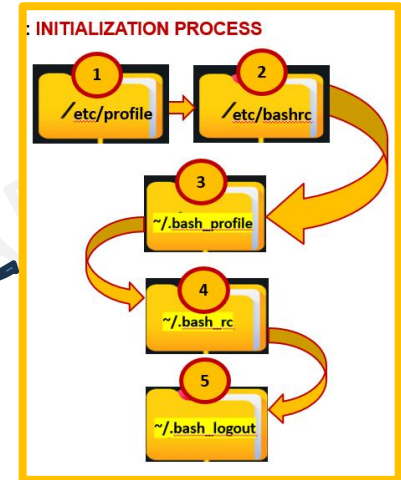
User

- 1 [root@PracticeLab ~]# **useradd tom**
- 2 [root@PracticeLab ~]# **cat /etc/passwd**

```
tom:x:1003:1003::/home/tom:/bin/bash
```

(username) (uid) (gid) (Home directory) (shell)

→ /etc/shadow file
- 3 [root@PracticeLab ~]# **cat /etc/shadow**
- 4 [root@PracticeLab ~]# **passwd tom**
- 5 [root@PracticeLab ~]# **pwck** (checks “/etc/passwd & /etc/shadow file integrity)



Commands

#useradd

#id <user> => (Check if user is added)

#passwd => (Create a user password)

#pwck => (check integrity of password)

OVERVIEW OF FILES RELATED TO USER MANAGEMENT

Files Related to User Management = 9

/etc/passwd	=> stores user information (username, uid)
/etc/shadow	=> stores user's password encrypted form
/etc/group	=> Stores group's information
/etc/passwd-	=> backup copy
/etc/shadow-	=> backup copy
/etc/group-	=> backup copy
/etc/default/useradd	=> user's login
/etc/login.defs	=> Change user aging policy and umask value
/etc/skel	=> initialization files / environmental variables

Command Related to User Management

#usermod

ADDING A NEW USER

When we add a user:

```
[root@Practicelab ~]# useradd TJ3211
```

Examining all the files when a User is added:

```
[root@Practicelab ~]# cat /etc/passwd
```

```
[root@Practicelab ~]# cat /etc/shadow
```

```
[root@Practicelab ~]# cat /etc/group
```

```
[root@Practicelab ~]# cat /etc/default/useradd
```

```
[root@Practicelab ~]# cat /etc/login.defs
```

```
[root@Practicelab ~]# cat /etc/skel
```

USERADD PRACTICE



Simplified steps to add a user in Linux are:

Step 1: Creating User account

#useradd [username]

```
root@Prod102:~  
[root@Prod102 ~]# useradd zawar
```

Step 2: adding a password to the users account

#passwd [username]

```
root@Prod102:~  
[root@Prod102 ~]# passwd zawar  
Changing password for user zawar.  
New password:  
BAD PASSWORD: The password is shorter than 8 characters  
Retype new password:  
passwd: all authentication tokens updated successfully.  
[root@Prod102 ~]#
```

WHAT HAPPENS WITH USERADD?

#**useradd** command does the following:

1. Add entries in user management files

- passwd (/etc/passwd)
- group (/etc/group)
- shadow (/etc/shadow)
- gshadow (/etc/gshadow)

```
[root@VM01 /]# cat /etc/default/useradd
# useradd defaults file
GROUP=100
HOME=/home
INACTIVE=-1
EXPIRE=
SHELL=/bin/bash
SKEL=/etc/skel
CREATE_MAIL_SPOOL=yes
```

2. Gets defaults from /etc/default/useradd

3. Creates a home directory for user /home/username

4. Copies initial configuration files from default skeleton directory /etc/skel into the user's home directory

```
root@Prod102:/etc/skel

[root@Prod102 ~]# cd /etc/skel
[root@Prod102 skel]# ls -al
total 24
drwxr-xr-x.  2 root root  62 Oct 13 12:43 .
drwxr-xr-x. 109 root root 8192 Jan  1 22:43 ..
-rw-r--r--.  1 root root  18 Mar 31  2020 .bash_logout
-rw-r--r--.  1 root root 193 Mar 31  2020 .bash_profile
-rw-r--r--.  1 root root 231 Mar 31  2020 .bashrc
```

CHECK ADDED USER STATUS



Simplified steps check new users status:

Step 1: extract user information from “passwd” file located in /etc directory

#grep [username] /etc/passwd

```
root@Prod102:~  
[root@Prod102 ~]# grep zawar /etc/passwd  
zawar:x:1005:5004::/home/zawar:/bin/bash
```

Default login shell
(-s) option to change

“zawar ali”
(-c) comment option

Base directory
(-b) to change

Step 2: extract user group info from “group” file located in /etc directory

#grep [username] /etc/group

```
root@Prod102:~  
[root@Prod102 ~]# grep zawar /etc/group  
zawar:x:5004:
```

Primary group name by default
(-g) option

KNOW YOUR SHELL



Check User Shell:

Step 1: extract user information from “passwd” file located in /etc directory

#grep [username] /etc/passwd

```
root@Prod102:~
```

```
[root@Prod102 ~]# grep zawar /etc/passwd
zawar:x:1005:5004::/home/zawar:/bin/bash
```

Default login shell

Valid Login Shells:

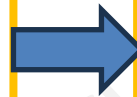
```
[nitadmin2@VM01 ~]$ cat /etc/shells
```

/bin/sh => **Unix Shell**

/bin/bash => **Linux Shell**

/usr/bin/sh

/usr/bin/bash



Run the following command to download “chsh”

```
[nitadmin2@VM01 ~]$ dnf provides "*/chsh"
```

```
Rocky Linux 9 - BaseOS 1.5 MB/s | 2.3
```

```
MB 00:01
```

```
Rocky Linux 9 - AppStream 3.6 MB/s |
```

```
8.4 MB 00:02
```

```
Rocky Linux 9 - Extras 15 kB/s | 16 kB
```

```
00:01
```

```
util-linux-user-2.37.4-20.el9.x86_64 : libuser based
```

```
util-linux utilities
```

```
Repo : baseos
```

```
Matched from:
```

```
Filename : /etc/pam.d/chsh
```

```
Filename : /usr/bin/chsh
```

```
Filename : /usr/share/bash-completion/completions/chsh
```


OPTIONS EXPLORED



Various options and how they can be used:

Option	How an 'OPTION' is used	comments
-b	#useradd -b /UAT zawar	Changing base directory #mkdir /UAT
-c	#useradd -c 'zawar ali' zawar	Adding comments
-g	#useradd -g devops zawar	Added to a primary group
-G	#useradd -G admin zawar	Added to a secondary group
-r	Used in special cases for services	
-s	#useradd -s /bin/tcsh zawar	Assign a new Shell
-u	#useradd -u 1001 zawar	Creating CUSTOM UID Default is the next available UID from /etc/passwd file. For example last used UID in /etc/passwd file is 1050, then UID 1051 will be used for this account.

PASSWORD



How do you know a password is assigned or not:

Step 1: check using grep command

#grep [username] /etc/shadow

```
root@Prod102:~  
[root@Prod102 ~]# grep ramsha /etc/shadow  
ramsha:!!18992:0:99999:7:::
```

Since no password is set, account is locked!!

Step 2: set passwd and grep to see the encrypted passwd

#grep [username] /etc/shadow

```
root@Prod102:~  
[root@Prod102 ~]# passwd ramsha  
Changing password for user ramsha.  
New password:  
BAD PASSWORD: The password is shorter than 8 characters  
Retype new password:  
passwd: all authentication tokens updated successfully.  
[root@Prod102 ~]# grep ramsha /etc/shadow  
ramsha:$6$diNsqNv3$UvAAzmQpAmmPmZleDYKpiYJBcfX2o.zMwpyGnChkhf5rYbwLCiOayqp.Dx6Gq2I6DNcO67pBqbQFycKsxv4qT1:18992:0:99999:7:::  
[root@Prod102 ~]#
```

PASSWORD - CHECK



I am trying to determine if the passwd and group files present in the location `/etc/passwd` in Linux contains **any invalid data entries**

```
root@Prod102:~  
[root@Prod102 ~]# pwck #pwck  
user 'cockpit-ws': directory '/nonexisting' does not exist  
user 'docker': program '/user/sbin/nologin' does not exist  
no matching password file entry in /etc/passwd  
delete line 'ftp*:18353:0:99999:7:::'? ^C  
[root@Prod102 ~]# grpck #grpck  
no matching group file entry in /etc/group  
delete line 'devops-secondary:!:ramsha'? █
```

PASSWORD (LOCK/UNLOCK)



Users can be locked and then unlocked in Linux:

```
root@Prod102:~  
[root@Prod102 ~]# grep ramsha /etc/shadow  
ramsha:$6$diNsQNV3$UvAAzmQpAmmPmZleDYKpiYJBcfX2o.zMwpyGnChkhf5rYbwLCiOayqp.Dx6Gq2  
[root@Prod102 ~]# usermod -L ramsha lock  
[root@Prod102 ~]# grep ramsha /etc/shadow  
ramsha:!  
[root@Prod102 ~]# usermod -U ramsha ←Unlock  
[root@Prod102 ~]# grep ramsha /etc/shadow  
ramsha:$6$diNsQNV3$UvAAzmQpAmmPmZleDYKpiYJBcfX2o.zMwpyGnChkhf5rYbwLCiOayqp.Dx6Gq2
```

USERMOD



Let us add description to a user

```
root@Prod102:~  
[root@Prod102 ~]# grep zavar /etc/passwd  
zavar:x:1005:5004::/home/zavar:/bin/bash  
[root@Prod102 ~]# usermod -c "zavar ali" zavar  
[root@Prod102 ~]# grep zavar /etc/passwd  
zavar:x:1005:5004:zavar ali:/home/zavar:/bin/bash
```

```
[root@Prod102 ~]# usermod -c 'zavar ali' zavar  
usermod: no changes
```

```
[root@Prod102 ~]# usermod -c 'zavar tiger' zavar  
[root@Prod102 ~]# grep zavar /etc/passwd  
zavar:x:1005:5004:zavar tiger:/home/zavar:/bin/bash
```

USERMOD OPTIONS EXPLORED



Various options and how they can be used:

Option	How an 'OPTION' is used	comments
-b	#usermod -b /TEST zawar	Changing base directory #mkdir /TEST
-c	#usermod -c 'zawar ali' zawar	Modify comments
-g	#usermod -g devops zawar	modify primary group
-G	#usermod -G admin zawar	User will be added in the secondary group but at the same time it will remove user from all existing groups
-aG	#usermod -aG admin zawar	User will be added in the secondary group but at the same time it will keep user in` all existing groups
-s	#usermod -s /bin/tcsh zawar	Modify a new Shell
-md	#usermod -md /home	
-l	#usermod -l zawar.ali zawar	Change username

DELETE A USER

Option 1: Delete the USER but KEEP their files (safe):

```
[root@Practicelab ~]# userdel username
```

Note: replace “username” with actual name!

Option 2: Delete the user AND their files (Permanent)

```
[root@Practicelab ~]# userdel -r username
```

User is currently used by a process...

```
[root@Practicelab ~]# killall -u username
```

```
[root@Practicelab ~]# userdel -r username
```

How to verify the User is removed from the system?

```
[root@Practicelab ~]# grep username /etc/passwd
```

USERDEL / RMDIR

By default, `userdel` retains the user's home directory. If you want to keep the home directory intact, omit the `-r` option.

NOTE: #userdel command will not remove users home directory

1. `#rmdir dir1`
2. `#rm -rf file1`

Option	Description
<code>-f</code>	Force removal of the user account, including home directory and mail spool, even if the user is logged in.
<code>-r</code>	Remove the user's home directory along with the account. Useful for a complete cleanup.
<code>-v</code>	verbose

OVERVIEW OF COMMANDS RELATED TO GROUPS

Total of 10 commands

#groupadd

#passwd

#groupdel

#groupdel -R

#groupmod

#grpck => Checks integrity and consistency of the /etc/group

#echo \$?

#grpconv => How to restore /etc/gshadow file if deleted by mistake

#chgrp

#chown (37 page 10)

#chage => information in /etc/shadow file will be modified

HOW DO YOU ADD A GROUP



Very simple!

```
root@Prod102:~  
[root@Prod102 ~]# groupadd hostinggp  
[root@Prod102 ~]# grep hostinggp /etc/group  
hostinggp:x:5007:
```

```
root@Prod102:~  
[root@Prod102 ~]# groupadd backup  
[root@Prod102 ~]# grep backup /etc/group  
backup:x:5008:
```

ADD USER TO GROUPS

Simplified steps to add a user in Linux are:

Step 1: Create couple of groups

#groupadd [group name]

```
root@Prod102:/home
[root@Prod102 home]# groupadd nexus-primary
[root@Prod102 home]# grep nexus-primary /etc/group
nexus-primary:x:5005:
[root@Prod102 home]# groupadd devops-secondary
[root@Prod102 home]# grep devops-secondary /etc/group
devops-secondary:x:5006:
```

Step 2: add a new user to an existing primary and secondary group(s)

**#useradd -c "ramsha babar" -g [primary group] -G [secondary group]
ramsha**

```
root@Prod102:~
[root@Prod102 ~]# useradd -c "ramsha babar" -g nexusprimary -G devsecondary ramsha
```

Step 3: modify an existing user and add him to a new secondary group

#usermod -aG groupname user

USER AND GROUP PRACTICE

```
[root@vm1~]# useradd testuser  
[root@vm1~]# tail -l /etc/passwd  
[root@vm1~]# tail -l /etc/group  
[root@vm1~]# groupadd testgroup  
[root@vm1~]# usermod -G testgroup tester  
[root@vm1~]# tail -l /etc/passwd  
[root@vm1~]# tail -l /etc/group
```

CHAGE

```
[tom@System-Admin product]$ chage -l tom
Last password change                : Jan 18, 2025
Password expires                    : never
Password inactive                   : never
Account expires                    : never
Minimum number of days between password change : 0
Maximum number of days between password change : 99999
Number of days of warning before password expires : 7
```

Minimum number of days between password change

```
[root@System-Admin tmp]# chage -m 5 tom
```

Maximum number of days between password change

```
[root@System-Admin tmp]# chage -M 90 tom
```

Number of days of **warning** before password expires

```
[root@System-Admin tmp]# chage -w 10 tom
```

Password inactive

```
[root@System-Admin tmp]# chage -I 5 tom
```

Account expires

```
[root@System-Admin tmp]# chage -E 2025-04-01
```

Password expires

```
[root@System-Admin tmp]# chage -d 2025-29-03
```

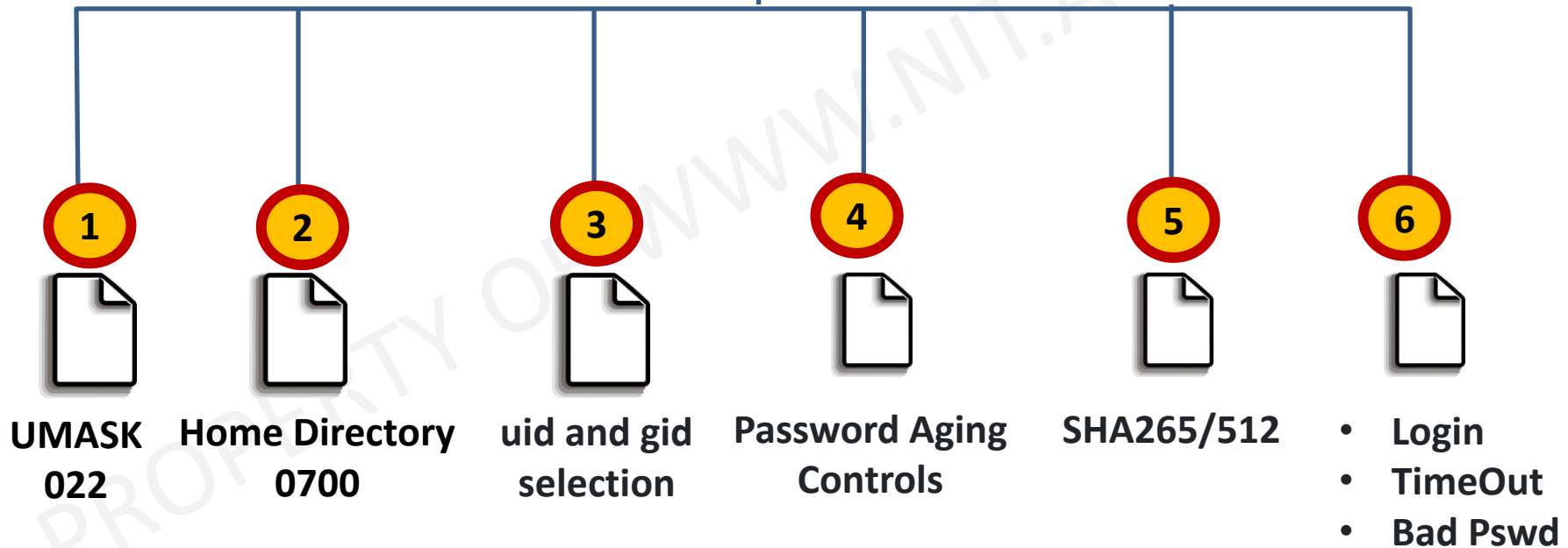
CHAGE - SUMMARY

Key Password Aging Fields in `/etc/shadow`

- **Last Password Change:** When the password was last changed (in days since the epoch).
- **Minimum Password Age:** The minimum number of days a password must be used before it can be changed.
- **Maximum Password Age:** The maximum number of days a password can be used before it needs to be changed.
- **Password Warning Period:** The number of days before a password expires when the user will be warned.
- **Password Inactivity Period:** The number of days after a password expires that the account will be disabled.
- **Account Expiration:** The date when the account will be disabled (optional).

DETAILED USER CONFIGURATION

```
$cat /etc/login.defs
```



DETAILED USER CONFIGURATION

```
[root@Practicelab ~]# lslogins root
```

```
Username:          root
UID:               0
Gecos field:       root
Home directory:    /root
Shell:             /bin/bash
No login:          no
Password is locked: no
Password not required (empty): no
Login by password disabled: no
Password encryption method: SHA-512
Primary group:     root
GID:              0
Last login:        07:29
Last terminal:     pts/0
Last hostname:     192.168.1.215
Hushed:           no
Password expiration warn interval: 7
Password changed:  May09/19:00
Maximum change time: 99999
Running processes: 101
```

Last logs:

```
09:23 NetworkManager[774]: <error> [1781360633.4081] dhcp6 (enX0): DAD failed for address:
2600:1702:54a7:3a10::1f/128 lft 3600sec pref 300sec lifetime 206076-206076[300,3600] dev 2 src dhcp
09:26 NetworkManager[774]: <info> [1781360772.7524] dhcp6 (enX0): state changed new lease,
address=2600:1702:54a7:3a10::1f
09:26 NetworkManager[774]: <error> [1781360772.7841] dhcp6 (enX0): DAD failed for address:
2600:1702:54a7:3a10::1f/128 lft 3600sec pref 300sec lifetime 206216-206216[300,3600] dev 2 src dhcp
```